

Skill 清单

版本：v1.0 | 日期：2026-08-05

本文件为「研发缺陷闭环协同系统」全部 Skill 的统一清单。依据 方案设计.md v2.0 §3.2 提炼，覆盖 15 个 AgentTeams Skills（6 个 prompt-only + 9 个 with-scripts）。

使用方式：– 新增 Skill 时，复制「空白模板」章节，按字段填写完整。– 修改 Skill 行为时，同步更新本文件对应条目，并标注变更版本。– 跨角色协作时，本文件是「我手里有什么 Skill、能用来做什么、需要什么前置条件」的统一参考。

一、字段定义

下表为每个 Skill 条目必须包含的 10 个字段。所有字段均为必填，“无”或“暂未启用”需显式标注。

字段	填写说明
Skill 名称	Skill 的名称或代号。命名规则：小写英文短词 + 连字符（如 code-search / lesson-lookup），与 deploy/packages/rd-defect-skills/skills/<name>/SKILL.md 目录名保持一致。
Skill 类型	官方云 Skill / 自定义 Skill / 外部工具封装 / 企业系统集成能力等。本系统全部为 AgentTeams 自定义 Skills，再细分 prompt-only（仅封装使用规范）和 with-scripts（含 scripts/ 脚本）。
使用场景	说明该 Skill 适用于哪类任务或环节。具体到哪个角色在哪个阶段使用。
输入参数	说明调用该 Skill 需要哪些输入。逐条列出字段、来源、格式要求。
输出结果	说明该 Skill 输出哪些结构化或非结构化结果。明确格式（JSON / Markdown / diff / 字符串）。
调用条件	说明在什么条件下触发调用。包括状态机阶段、角色、前置 Skill 完成情况。

字段	填写说明
依赖工具/系统	说明该 Skill 依赖的云产品、企业系统、MCP 工具、数据库或外部 API。
失败处理	说明调用失败、结果异常或超时时处理方式。明确降级策略与升级路径。
权限与安全	说明调用过程中的权限控制、密钥管理和风险边界。
复用价值	说明该 Skill 是否可在其他 Agent 或其他场景中复用。

二、Skill 分类总览

类型	数量	Skill 列表
prompt-only	6	code-read、bash-exec、test-run、patch-generate、result-judge、multi-file-edit
with-scripts	9	repo-index、code-search、impact-analysis、root-cause-analysis、repair-planning、pipeline-router、module-lookup、lesson-lookup、knowledge-extraction
合计	15	—

全部 Skill 打包在 `deploy/packages/rd-defect-skills-vX.Y.Z.zip` 中，通过 Worker YAML 的 `spec.package` 字段 pinned 版本引用。

三、prompt-only Skills（6 个）

由 Worker runtime 直接提供执行能力，Skill 仅封装使用规范（含 prompt 指令与 LLM 使用约定）。

3.1 code-read

字段	内容
Skill 名称	code-read
Skill 类型	AgentTeams Skills – prompt-only
使用场景	读取代码文件，支持指定行范围与上下文窗口。 Analyzer 阅读源码、Fixer 读取待修改文件、 Evaluator 审查 diff 时使用。
输入参数	① file_path（必填，仓库内相对路径） ② start_line / end_line（可选，行号范围） ③ context_window（可选，默认 ±10 行）
输出结果	文件内容（含行号标记的字符串），超出范围时按窗口截断并提示。
调用条件	任何需要读取源码的环节均可调用；Fixer 调用时需先通过 pipeline-router 确认本轮编辑范围。
依赖工具/系统	Worker 原生工具 file r/w（只读模式）。
失败处理	文件不存在 → 返回错误码 + 建议路径；权限拒绝 → escalate 给 Manager；编码异常 → 自动尝试 UTF-8 / GBK 降级。
权限与安全	沙箱路径限制（仅仓库工作区内）；只读访问；不返回 .env / secrets/ 等敏感目录。
复用价值	高。Manager/Analyzer/Fixer/Evaluator 四个角色均使用，可推广到任何“阅读代码”场景。

3.2 bash-exec

字段	内容
Skill 名称	bash-exec
Skill 类型	AgentTeams Skills – prompt-only
使用场景	安全执行 bash 命令。Tester 执行 pytest、Fixer 跑 git diff、Manager 调外部脚本、Analyzer 运行复现脚本。

字段	内容
输入参数	① <code>command</code> (必填, shell 命令字符串) ② <code>cwd</code> (可选, 工作目录) ③ <code>timeout_sec</code> (可选, 默认 60s, 上限 600s) ④ <code>env_overrides</code> (可选, 临时环境变量)
输出结果	<code>stdout</code> / <code>stderr</code> / <code>exit_code</code> / <code>duration_ms</code> 四个字段。
调用条件	任何需要执行 shell 命令时; 白名单外的命令直接拒绝。
依赖工具/系统	Worker 原生工具 <code>bash</code> (带白名单 + 超时)。
失败处理	白名单拦截 → 返回拒绝原因; 超时 → SIGTERM → 强制 kill 进程; 非零退出码 → 返回完整 <code>stderr</code> 供调用方判断。
权限与安全	白名单机制 (仅允许安全命令如 <code>git</code> / <code>pytest</code> / <code>pip</code> / <code>python</code> 等); 超时强制终止; 禁止 <code>rm -rf</code> / <code>sudo</code> / 反弹 shell 等高危命令; 不继承宿主机环境变量。
复用价值	高。五个角色均使用, 是事实上的”通用执行入口”。

3.3 test-run

字段	内容
Skill 名称	<code>test-run</code>
Skill 类型	AgentTeams Skills – prompt-only
使用场景	执行测试并解析 <code>pytest</code> 输出。Tester 真实测试执行阶段。
输入参数	① <code>test_cmd</code> (默认 <code>pytest -v --tb=short</code>) ② <code>target_files</code> (可选, 限定测试文件) ③ <code>extra_args</code> (可选, 如 <code>-k pattern</code>)
输出结果	<code>test_report.json</code> : 含 <code>passed_cases</code> / <code>failed_cases</code> / <code>error_type</code> / <code>traceback</code> / <code>failing_line</code> / <code>duration_sec</code> 。
调用条件	Tester 进入 <code>testing</code> 状态、已应用 <code>fix.diff</code> 、 <code>venv</code> 已搭建完成后触发。

字段	内容
依赖工具/系统	<code>bash-exec</code> 、Python <code>venv</code> 、 <code>pytest</code> 。
失败处理	<code>pytest</code> 解析失败 → 返回原始 <code>stdout/stderr</code> ；环境异常（缺包/版本冲突）→ <code>escalate</code> ；同一 <code>diff</code> 连续 3 轮失败 → <code>escalate</code> 。
权限与安全	在隔离 <code>venv</code> 内执行，不污染主环境；超时 10 分钟强制终止；不联网下载未声明依赖。
复用价值	中。主要 <code>Tester</code> 使用；扩展多语言时复用同一执行框架（Java/JS/Go 各实现一层）。

3.4 patch-generate

字段	内容
Skill 名称	<code>patch-generate</code>
Skill 类型	AgentTeams Skills – prompt-only
使用场景	生成统一格式 <code>diff</code> 。 <code>Fixer</code> 完成多文件编辑后，汇总产出 <code>fix.diff</code> 。
输入参数	① <code>repo_path</code> （仓库根路径） ② <code>base_commit</code> （基准 <code>commit</code> SHA） ③ <code>target_files</code> （修改文件列表，可选，默认全部）
输出结果	<code>unified diff</code> 格式字符串（ <code>--- a/... / +++ b/...</code> 头），写入 <code>fix.diff</code> 文件。
调用条件	<code>Fixer</code> 完成所有 <code>edit_file</code> 调用后、 <code>pipeline-router</code> 状态推进到 <code>testing</code> 之前调用。
依赖工具/系统	Worker 原生工具 <code>git diff</code> 。
失败处理	无 <code>git</code> 仓库 → 返回错误；非 UTF-8 文件 → 提示二进制文件单独处理；编码异常 → 回退到文件级 <code>diff</code> 。
权限与安全	只读访问 <code>git</code> 历史；不修改任何文件；不暴露敏感信息（如密钥、token）。
复用价值	中。主要 <code>Fixer</code> 使用；任何需要产出“代码变更”产物的场景可复用。

3.5 result-judge

字段	内容
Skill 名称	result-judge
Skill 类型	AgentTeams Skills – prompt-only
使用场景	基于结果裁定通过/驳回。Tester 反馈失败时识别”是否值得重试”、Evaluator 四维度审查。
输入参数	① artifact (待审查产物, 如 fix.diff + test_report.json) ② criteria (评分维度: 正确性/完整性/一致性/质量) ③ threshold (通过阈值)
输出结果	verdict.json: decision (pass/reject) / scores (四维度明细) / reasons (驳回原因)。
调用条件	Tester 完成一轮测试后、Evaluator 进入 evaluating 阶段时触发。
依赖工具/系统	无外部依赖 (纯 LLM 推理)。
失败处理	四维度评分内部矛盾 (如正确性高分但质量低分) → escalate; 评分置信度 < 0.6 → escalate。
权限与安全	仅生成判定, 不修改任何数据; 不读取敏感文件。
复用价值	高。Tester/Evaluator 共用, 可推广到任何”质量评估”环节 (如 PR 审查、文档质量检查)。

3.6 multi-file-edit

字段	内容
Skill 名称	multi-file-edit
Skill 类型	AgentTeams Skills – prompt-only
使用场景	多文件协调编辑。Fixer 单轮需修改多文件时, 按 repair-planning 输出的方案顺序编辑。
输入参数	① edit_plan (文件路径 + 目标修改的列表) ② atomic (是否原子化, 默认 true)

字段	内容
输出结果	各文件编辑结果（成功/失败/行号变化）、整体一致性报告。
调用条件	Fixer 处于 <code>fixing</code> 状态、 <code>repair-planning</code> 已输出方案、文件数 ≤ 5 。
依赖工具/系统	Worker 原生工具 <code>edit_file</code> 、 <code>git</code> 。
失败处理	单文件失败 $\rightarrow$ <code>atomic=true</code> 时回滚所有文件；记录失败文件供下一轮 <code>repair-planning</code> 调整方案。
权限与安全	沙箱路径限制；禁止修改 <code>.git/</code> / <code>.env</code> / <code>secrets/</code> ；编辑前自动备份（ <code>*.bak</code> ）。
复用价值	中。主要 Fixer 使用；任何”批量修改代码”场景可复用。

四、with-scripts Skills（9 个）

Skill 内部通过 `scripts/*.py` 脚本编排 MCP 原语，完成特定工作流。脚本可独立测试、可显式埋 OTel Span。

4.1 repo-index

字段	内容
Skill 名称	<code>repo-index</code>
Skill 类型	AgentTeams Skills – with-scripts（核心脚本： <code>index.py</code> ）
使用场景	增量索引：分块 $\rightarrow$ 嵌入 $\rightarrow$ 三库写入（ <code>pgvector</code> + <code>Neo4j</code> + <code>Meilisearch</code> ）。Manager 首次全量索引 + 后续增量更新。
输入参数	① <code>repo_path</code> ② <code>repo_name</code> ③ <code>commit_sha</code> ④ <code>full_reindex</code> （布尔，默认 <code>false</code> ）

字段	内容
输出结果	索引统计 (<code>chunk_count</code> / <code>file_count</code> / <code>duration_sec</code>)、Redis 中更新 <code>repo_state</code> hash、嵌入缓存命中率。
调用条件	Manager 收到新任务且 Redis 中 <code>repo_state</code> 与目标 <code>commit</code> 不一致时触发。
依赖工具/系统	MCP 原语: <code>pgvector_upsert_chunk</code> / <code>meili_keyword_search</code> (写入端) / <code>neo4j_*</code> (图谱写入) / <code>embed_texts</code> / <code>redis_get_repo_state</code> / <code>redis_set_repo_state</code> / <code>redis_fetch_embedding</code> / <code>ast_parse_file</code> 。 数据库: <code>pgvector</code> 、 <code>Neo4j</code> 、 <code>Meilisearch</code> 、 <code>Redis</code> 。
失败处理	单文件嵌入失败 → 跳过该文件、记录 <code>failed_files</code> 、下次增量补充；三库部分失败 → 按库独立重试，最终一致性；P99 目标 ≤ 60s, 超时则降级为分批处理。
权限与安全	数据库连接凭证通过 <code>deploy/install/agentteams.env</code> 注入；输入校验 (路径防穿越)；审计日志记录每次索引操作。
复用价值	中。仅 Manager 在调度初期使用；可作为独立”仓库索引”服务推广到其他需要代码检索的场景。

4.2 code-search

字段	内容
Skill 名称	<code>code-search</code>
Skill 类型	AgentTeams Skills – with-scripts (核心脚本: <code>context_packer.py</code>)
使用场景	语义+关键词搜索+结果打包。Analyzer 根因分析、Manager 初步检索时使用。

字段	内容
输入参数	① query_text ② top_k (默认 10) ③ repo_filter (可选) ④ strategy (semantic / keyword / hybrid, 默认 hybrid)
输出结果	结构化 Top-K 结果: file_path / line_range / score / snippet / context_window。
调用条件	Analyzer 进入 analyzing 阶段、Manager 调度初期需对 Issue 做初步匹配时触发。
依赖工具/系统	MCP 原语: hybrid_search (共享服务, RRF 融合) / pgvector_search / meili_keyword_search / neo4j_expand_chunks (对融合后 chunks 做调用链扩展) / embed_texts。 脚本: context_packer.py (打包为带上下文的结构化结果)。
失败处理	召回为空 → 降级为纯关键词搜索; RRF 融合超时 → 返回单路 Top-K; Neo4j 扩展失败 → 仅返回融合结果, 标记“无调用链上下文”。
权限与安全	只读数据库查询; 输入 query 长度限制 ($\leq 2\text{KB}$); 审计日志记录每次查询的 query_hash 与命中数。
复用价值	高。Analyzer / Fixer / Manager 均可调用; 可作为通用“代码语义搜索”服务暴露给其他系统。

4.3 impact-analysis

字段	内容
Skill 名称	impact-analysis
Skill 类型	AgentTeams Skills – with-scripts (核心脚本: dep_graph.py / contract_check.py)
使用场景	依赖图分析与契约检查。Analyzer 评估影响面、Evaluator 审查一致性时使用。

字段	内容
输入参数	① <code>target_symbols</code> （函数/类/模块列表） ② <code>depth</code> （图遍历深度，默认 2） ③ <code>check_contract</code> （布尔，是否检查 API 契约）
输出结果	影响面统计（ <code>affected_files</code> / <code>affected_functions</code> / <code>affected_callers</code> ）、契约违规列表（破坏的接口签名、不兼容的返回类型）。
调用条件	Analyzer 完成根因定位后、Fixer 制定修复方案前；Evaluator 审查 patch 时再次确认影响面。
依赖工具/系统	MCP 原语： <code>neo4j_impact_stats</code> / <code>neo4j_dep_subgraph</code> / <code>neo4j_symbol_lookup</code> / <code>ast_parse_file</code> 。 数据库：Neo4j 图谱。
失败处理	图谱数据缺失 → 返回空集 + 提示”需先 <code>repo-index</code> “；契约解析失败 → 跳过该模块并记录警告。
权限与安全	只读 Neo4j；输入符号白名单（仅仓库内符号）；超时 30s。
复用价值	高。Analyzer / Evaluator 共用；可推广到”代码变更风险评估”通用场景。

4.4 root-cause-analysis

字段	内容
Skill 名称	<code>root-cause-analysis</code>
Skill 类型	AgentTeams Skills – with-scripts（核心脚本： <code>root_cause.py</code> ）
使用场景	根因推断与复现脚本生成。Analyzer 在 <code>analyzing</code> 阶段整合多源信息后，产出结构化根因报告。
输入参数	① <code>issue_text</code> ② <code>search_results</code> （来自 <code>code-search</code> ） ③ <code>impact_analysis</code> （来自 <code>impact-analysis</code> ） ④ <code>historical_lessons</code> （来自 <code>lesson-lookup</code> ）

字段	内容
输出结果	<code>analysis_report.json</code> （结构化根因 + 修复策略）+ <code>root_cause_report.md</code> （人类可读）+ 可选 <code>reproduce.py</code> （复现脚本）。
调用条件	Analyzer 完成 <code>code-search</code> / <code>impact-analysis</code> / <code>lesson-lookup</code> 三个前置 Skill 后调用。
依赖工具/系统	编排 <code>code-search</code> / <code>impact-analysis</code> / <code>lesson-lookup</code> Skill；MCP 原语： <code>embed_texts</code> （用于根因向量化）。
失败处理	根因不明确 → 输出 2~3 个候选根因 + 置信度；复现脚本生成失败 → 仅输出分析报告；置信度 < 0.5 → <code>escalate</code> 。
权限与安全	仅生成报告，无副作用；不修改任何文件；LLM 输出经格式校验。
复用价值	中。主要 Analyzer 使用；可作为“故障根因分析”通用能力推广到 SRE / 运维场景。

4.5 repair-planning

字段	内容
Skill 名称	<code>repair-planning</code>
Skill 类型	AgentTeams Skills – with-scripts（核心脚本： <code>repair_plan.py</code> / <code>risk_gate.py</code> ）
使用场景	修复方案规划与风险闸门。Fixer 在编辑前制定具体方案，并通过风险闸门决定是否执行。
输入参数	① <code>root_cause_report</code> ② <code>impact_analysis</code> ③ <code>historical_lessons</code> （ <code>fix_pattern</code> ）
输出结果	修复方案（ <code>file_edits</code> 列表，每项含文件路径、目标函数、修改类型、改动摘要）+ 风险等级（L0–L3）+ 闸门决策（ <code>pass</code> / <code>warn</code> / <code>block</code> ）。
调用条件	Fixer 进入 <code>fixing</code> 状态、Analyzer 已输出根因报告、Fixer 未做实际编辑之前。

字段	内容
依赖工具/系统	MCP 原语: <code>neo4j_impact_stats</code> (再确认影响面)。 脚本: <code>repair_plan.py</code> (LLM 推理生成方案) / <code>risk_gate.py</code> (规则化风险评估)。
失败处理	风险等级 $\geq$ L3 $\rightarrow$ block 并 escalate; 方案涉及 >5 文件 $\rightarrow$ 拆分多轮; <code>risk_gate</code> 与 <code>impact-analysis</code> 结论矛盾 $\rightarrow$ 保守侧生效。
权限与安全	仅生成方案, 无副作用; 风险等级评估由 <code>risk_gate.py</code> 规则化 (不依赖 LLM 主观判断); 不读取敏感文件。
复用价值	中。主要 Fixer 使用; 可推广到“代码变更方案设计”通用场景。

4.6 pipeline-router

字段	内容
Skill 名称	<code>pipeline-router</code>
Skill 类型	AgentTeams Skills – with-scripts (核心脚本: <code>task_router.py</code> / <code>state_manager.py</code> / <code>handoff.py</code> / <code>loop_judge.py</code>)
使用场景	任务路由、状态推进、人工交接。Manager 在每个阶段过渡时调用, 决定下游 Agent 与状态切换。
输入参数	① <code>task_id</code> ② <code>current_state</code> (TaskState) ③ <code>current_artifact</code> (当前阶段产物)
输出结果	派单指令 (<code>assignee</code> + <code>payload</code>) + 状态推进指令 (<code>next_state</code>) + 人工交接消息 (<code>matrix_message</code> , 可选)。
调用条件	Manager 每次收到 Worker 产出后、推进状态机前调用; <code>loop_judge</code> 检测到重复失败时触发人工交接。

字段	内容
依赖工具/系统	Matrix 联邦通信；TaskState 持久化（MinIO / Redis）；MCP 原语： <code>redis_set_repo_state</code> （状态快照）。 脚本： <code>task_router.py</code> （路由决策） / <code>state_manager.py</code> （状态机管理） / <code>handoff.py</code> （人工交接） / <code>loop_judge.py</code> （循环判定）。
失败处理	状态机冲突 → 回滚到上一稳定态；Matrix 通信失败 → 重试 3 次后 <code>escalate</code> ； <code>regression_cycle_count >= 3</code> → 强制 <code>escalated</code> 。
权限与安全	状态变更幂等（同一 <code>task_id</code> + 同一 <code>current_state</code> 不重复派单）；人工交接走 Matrix 加密通道；不暴露内部状态机细节给外部。
复用价值	中。仅 Manager 使用；可作为“多阶段任务调度”通用模式推广到其他多 Agent 协作场景。

4.7 module-lookup

字段	内容
Skill 名称	<code>module-lookup</code>
Skill 类型	AgentTeams Skills – with-scripts（核心脚本： <code>module_lookup.py</code> ）
使用场景	模块定位与入口点发现。Analyzer 在初步分析时快速定位“Issue 涉及哪个模块、入口在哪”。
输入参数	① <code>keyword</code> （模块名/关键词/Issue 摘要） ② <code>repo_filter</code> （可选） ③ <code>top_k</code> （默认 5）
输出结果	候选模块列表： <code>module_path</code> / <code>entry_points</code> （入口函数/类） / <code>related_symbols</code> / <code>score</code> 。
调用条件	Analyzer 在调用 <code>code-search</code> 之前做“模块级预筛”时触发；Manager 调度初期也可用。

字段	内容
依赖工具/系统	MCP 原语: <code>neo4j_symbol_lookup</code> / <code>neo4j_dep_subgraph</code> / <code>pgvector_search</code> (语义匹配)。 数据库: Neo4j + pgvector。
失败处理	未找到 → 返回相邻模块候选 + 提示手动指定; 图谱缺失 → 降级为纯语义搜索。
权限与安全	只读 Neo4j + pgvector; 输入 query 长度限制 (≤ 1KB); 超时 15s。
复用价值	中。主要 Analyzer 使用; 可推广到“代码导航”通用工具。

4.8 lesson-lookup

字段	内容
Skill 名称	<code>lesson-lookup</code>
Skill 类型	AgentTeams Skills – with-scripts (核心脚本: <code>lesson_lookup.py</code> , v1.9 新增)
使用场景	按角色模式查询 <code>lessons</code> 历史经验并按相似度 score 分级。Analyzer 按根因维度查询 (mode=analyzer)、Fixer 按改法维度查询 (mode=fixer)。
输入参数	① <code>query_text</code> (Issue 描述 或 <code>fix_pattern</code> + <code>error_signature</code>) ② <code>mode</code> (<code>analyzer</code> / <code>fixer</code>) ③ <code>repo_filter</code> ④ <code>top_k</code> (analyzer 默认 5, fixer 默认 3) ⑤ <code>success_only</code> (fixer 模式默认 true)
输出结果	匹配 lessons 列表 (按 score 三级分流): <code>HIGH</code> (≥0.85) / <code>MEDIUM</code> (0.60~0.85) / <code>LOW</code> (<0.60), 每条含 <code>lesson_id</code> / <code>root_cause</code> / <code>fix_pattern</code> / <code>diff_summary</code> / <code>edge_cases</code> / <code>merge_count</code> 。
调用条件	Analyzer 在根因分析前/中调用 (mode=analyzer); Fixer 在生成 patch 前调用 (mode=fixer, 仅 <code>success = true</code>)。

字段	内容
依赖工具/系统	MCP 原语: <code>pgvector_search</code> / <code>embed_texts</code> (query 向量化)。 数据库: PostgreSQL <code>lessons</code> 表 (pgvector 存储 <code>root_cause_vec</code>)。 脚本: <code>lesson_lookup.py</code> (业务编排: 相似度计算 + 分级 + 结果格式化)。
失败处理	召回为空 → 返回 LOW 级别空集, 提示”按标准流程执行”; 向量检索超时 → 降级为标签匹配; score 计算异常 → 标记为 LOW 保守处理。
权限与安全	只读 <code>lessons</code> 表; 输入 query 长度限制 ($\leq$ 2KB); 不返回 <code>merge_count < 1</code> 的低可信度记录; 不暴露 <code>success = false</code> 的失败案例给 Fixer。
复用价值	高。Analyzer / Fixer 共用, 是经验沉淀闭环的关键入口; 可推广到任何”基于历史经验做决策”的场景 (如客服 FAQ 推荐、运维故障处理)。

4.9 knowledge-extraction

字段	内容
Skill 名称	<code>knowledge-extraction</code>
Skill 类型	AgentTeams Skills – with-scripts (核心脚本: <code>extract.py</code>)
使用场景	从修复结果提取模式与经验。Evaluator 裁定完成后, 将本次修复的结构化经验写入 <code>lessons</code> 表。
输入参数	① <code>fix_diff</code> ② <code>test_report</code> ③ <code>verdict</code> ④ <code>root_cause_report</code> ⑤ <code>task_id</code> / <code>repo</code> / <code>retry_count</code> (来自 TaskState)

字段	内容
输出结果	新写入的 <code>lessons</code> 记录 (含 <code>id</code> / <code>root_cause</code> / <code>fix_pattern</code> / <code>error_signature</code> / <code>fix_strategy</code> / <code>affected_modules</code> / <code>tags</code> / <code>diff_summary</code> / <code>test_changes</code> / <code>edge_cases</code> / <code>success</code> / <code>resolution_summary</code> / <code>retry_count</code> / <code>merge_count</code> / <code>related_to</code>)。
调用条件	Evaluator 产出 <code>verdict.json</code> 后立即调用；不论 <code>pass/reject</code> 均触发 (失败经验同样有价值)。
依赖工具/系统	MCP 原语: <code>pgvector_search</code> (去重比对) / <code>pgvector_upsert_chunk</code> (写入) / <code>embed_texts</code> (向量化)。 数据库: PostgreSQL <code>lessons</code> 表。 脚本: <code>extract.py</code> (业务编排: 字段抽取 + 写入前去重 + MERGE/SIMILAR/NEW 决策 + 合并规则)。
失败处理	字段抽取失败 → 跳过该字段、记录 <code>incomplete_fields</code> ；写入前去重比对失败 → 保守标记为 <code>NEW</code> ；数据库写入失败 → 重试 3 次、最终失败时记录日志 (不阻塞主流程)。
权限与安全	写 <code>lessons</code> 表权限受最小化控制 (仅 Evaluator 角色持有写凭证)；输入校验 (防注入)；审计日志记录每次写入；写入前自动去重避免经验库膨胀。
复用价值	中。仅 Evaluator 使用；可作为“经验沉淀引擎”独立推广到其他需要“案例 → 知识”转化的场景。

五、Skill 依赖与调用矩阵

5.1 按角色 × Skill 矩阵

符号：● 主用 / ○ 备用 / — 不使用

Skill	Manager	Analyzer	Fixer	Tester	Evaluator
<code>code-read</code>	○	●	●	—	●

Skill	Manager	Analyzer	Fixer	Tester	Evaluator
bash-exec	●	●	●	●	●
test-run	—	—	—	●	—
patch-generate	—	—	●	—	—
result-judge	—	—	—	●	●
multi-file-edit	—	—	●	—	—
repo-index	●	—	—	—	—
code-search	○	●	○	—	—
impact-analysis	—	●	—	—	●
root-cause-analysis	—	●	—	—	—
repair-planning	—	—	●	—	—
pipeline-router	●	—	—	—	—
module-lookup	—	●	—	—	—
lesson-lookup	—	●	●	—	—
knowledge-extraction	—	—	—	—	●

5.2 Skill 编排关系 (with-scripts 之间)

root-cause-analysis

- └─ code-search (前置)
- └─ impact-analysis (前置)
- └─ lesson-lookup (前置, mode=analyzer)

repair-planning

- └─ impact-analysis (前置, 再确认)
- └─ lesson-lookup (前置, mode=fixer)

knowledge-extraction (Evaluator 内部)

- └─ 直接调用 MCP 原语 (pgvector_search / pgvector_upsert_chunk / embed_texts)

六、变更记录

版本	日期	变更内容	作者
v1.0	2026-08-05	初版：基于方案设计 v2.0 §3.2 提炼全部 15 个 Skill（6 prompt-only + 9 with-scripts）的完整清单	—